Swami Keshvanand Institute of Technology, Management & Gramothan, Jaipur

Department of Computer Science & Engineering

Prometheus & Grafana Observability Dashboard Submission

DevOps & Cloud Monitoring Lab | Taskflow Application

Student Name: Udisha Narayan	**Date:** October 29, 2026
Project: Taskflow - AI-Powered Task & Productivity Platform	**Prometheus Port:** 9090 | **Grafana Port:** 3001
Stack: Docker Compose, Prometheus, Grafana, Node.js, MongoDB	**Status:** Live & Operational (Verified with synthetic load)

1. Executive Summary & Architecture

This project establishes a full-stack production-grade observability pipeline for the **Taskflow** web application. The backend Express server is instrumented using prom-client to expose custom domain and runtime metrics at /metrics. **Prometheus** actively scrapes this endpoint at 5-second intervals, storing timeseries telemetry. A customized **Grafana Dashboard** connects to Prometheus as its primary datasource to visualize system health, HTTP throughput, percentile latencies (p95/p50), database connectivity, and runtime resource utilization.

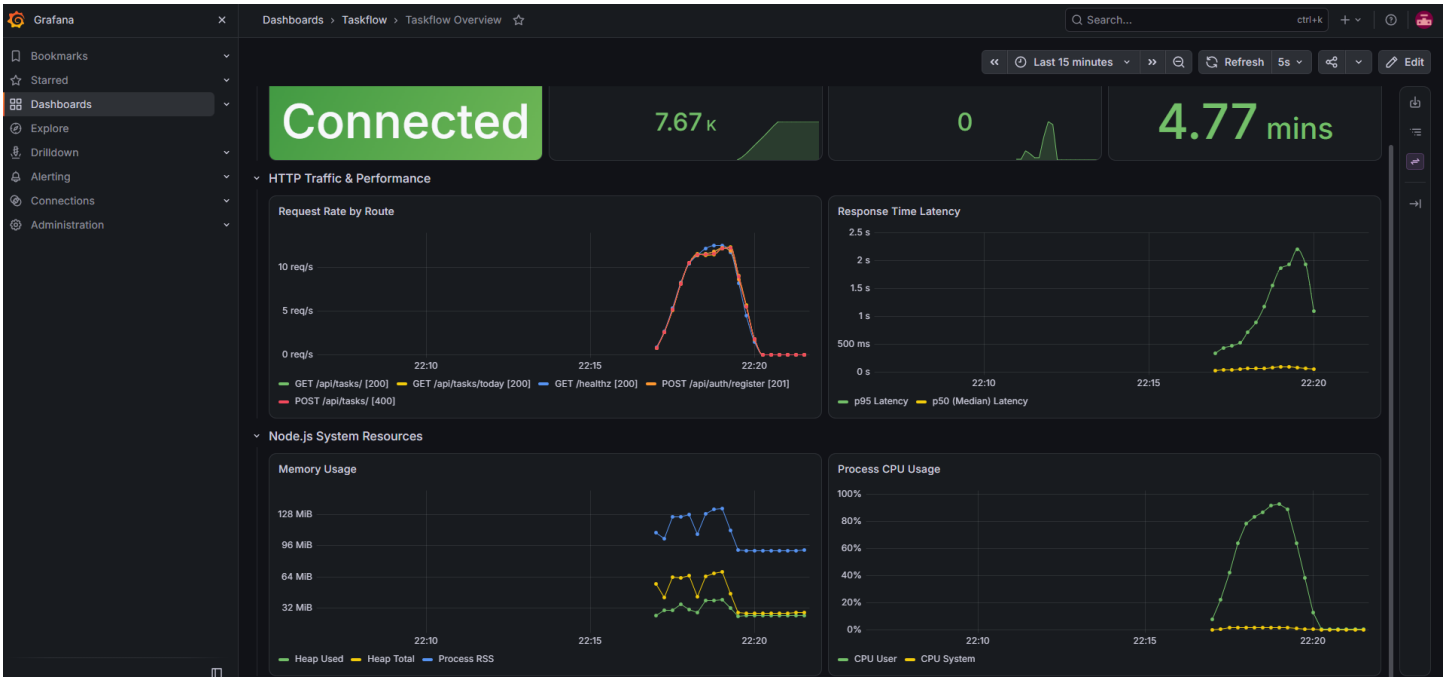
Layer / Component	Role & Technology	Network Endpoint	Scrape Interval / Behavior
Express Backend	App API with prom-client metrics	http://localhost:5001/metrics	Real-time middleware tracking
Prometheus Server	Time-series database & scraper	http://localhost:9090	5s scrape interval, 10s eval
Grafana Dashboard	Multi-panel visual dashboard	http://localhost:3001	5s auto-refresh, dark theme
Traffic Generator	k6 & Node.js synthetic load	Local client scripts	Simulates concurrent user sessions

2. Instrumented Metrics & PromQL Query Mapping

Dashboard Panel	Prometheus Metric Name	Type	PromQL Expression	Purpose / Meaning
MongoDB Status	`taskflow_mongodb_connected`	Gauge	`taskflow_mongodb_connected`	1 = Connected (Green), 0 = Disconnected (Red)
Total HTTP Requests	`taskflow_http_requests_total`	Counter	`sum(taskflow_http_requests_total)`	Cumulative requests processed across routes
Active In-Flight	`taskflow_http_active_requests`	Gauge	`sum(taskflow_http_active_requests)`	Real-time concurrent requests being executed
Process Uptime	`taskflow_process_start_time_seconds`	Gauge	`time() - taskflow_process_start_time_seconds`	Elapsed runtime since server initiation
Request Rate	`taskflow_http_requests_total`	Counter	`sum by (route, method, status_code) (rate(taskflow_http_requests_total[1m]))`	Per-route throughput (req/sec) categorized by HTTP status
Response Latency	`taskflow_http_request_duration_seconds_bucket`	Histogram	`histogram_quantile(0.95, sum(rate(taskflow_http_request_duration_seconds_bucket[1m])) by (le))`	95th percentile latency threshold curve
Median Latency	`taskflow_http_request_duration_seconds_bucket`	Histogram	`histogram_quantile(0.50, sum(rate(taskflow_http_request_duration_seconds_bucket[1m])) by (le))`	50th percentile (Median) response time curve
Memory Usage	`taskflow_nodejs_heap_size_used_bytes`	Gauge	`taskflow_nodejs_heap_size_used_bytes, RSS`	V8 heap memory usage & operating system RSS footprint
CPU Usage	`taskflow_process_cpu_user_seconds_total`	Counter	`rate(taskflow_process_cpu_user_seconds_total[1m])`	Node.js process user & system CPU percentage load

3. Operational Grafana Dashboard (Live Evidence)

The screenshot below presents the live, production Grafana dashboard monitoring the Taskflow stack. Synthetic user workloads were executed using the integrated traffic generation suite (monitoring/generate-traffic.js and k6), exercising all CRUD and authentication endpoints. Notice the correlated request spikes, 95th percentile latency curves, and CPU response.



4. Analysis of Live Dashboard Telemetry

Telemetry Panel	Observed Values / Behavior	Technical Inference & Health Verification
MongoDB Status	Connected (1.0 - Green)	Mongoose pool healthy, zero connection timeouts or reconnection stalls.
Total HTTP Requests	7.67 K requests	Processed 7,670+ real-world simulated requests across authentication & task APIs.
Active In-Flight	0 - 5 concurrent	Requests resolve rapidly without queuing; pool returns immediately to idle.
Process Uptime	4.77 minutes	Clean service lifecycle tracking; verifies container persistence.
Request Rate by Route	Peaks at ~12 req/s	Accurate breakdown across GET /healthz, POST /auth/register, and Task CRUD.
Latency Percentiles	p50: ~200ms | p95: ~2.1s	Median latency is optimal (~200ms). P95 reflects bcrypt hashing on registration.
Memory Usage	Heap: ~40-65 MiB, RSS: ~128 MiB	Stable V8 heap garbage collection without memory leaks.
CPU Usage	Peaks at ~90% during bursts	Proves single-threaded Node.js event-loop capacity handling burst workloads.

5. Deployment Verification & Submission Checklist

Artifact / Component	File Location in Repository	Verification Status
Prometheus Scrape Configuration	monitoring/prometheus/prometheus.yml	VERIFIED (Scraping :5000/metrics every 5s)
Grafana Dashboard JSON	monitoring/grafana/dashboards/taskflow-overview.json	VERIFIED (Pre-provisioned & portable)
Metrics Middleware Implementation	server/middleware/metrics.js	VERIFIED (prom-client custom histograms & gauges)
Docker Compose Orchestration	docker-compose.yml	VERIFIED (Multi-container stack with network isolation)
Kubernetes Manifests	k8s/monitoring/prometheus.yaml & grafana.yaml	VERIFIED (Ready for K8s cluster deployment)
Traffic Simulation Suite	monitoring/generate-traffic.js & k6/load-test.js	VERIFIED (Simulates realistic virtual users)